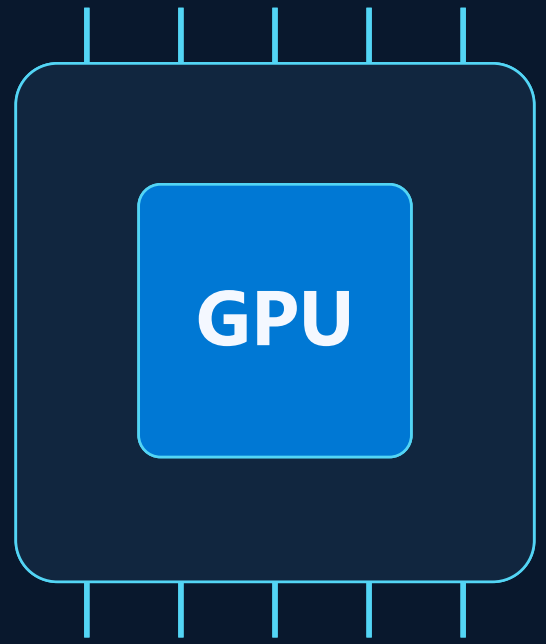
DEVELOPER & ARCHITECT BRIEFING

Azure Container Apps

Serverless GPU

Bring your model.
Keep the GPU. Lose the cluster management.



ON DEMAND

NVIDIA T4 / A100

ELASTIC

Scale to zero

USAGE BASED

Per-second compute

25-30 MINUTES | TECHNICAL OVERVIEW | 20 OCT 2026

Source: GPU overview

GPU acceleration without an always-on GPU fleet

Best fit: variable demand, custom inference, and teams that want less infrastructure to operate.

CONTROL

Bring your stack

Own the container image, model weights, inference runtime and API contract.

ELASTICITY

Follow demand

Scale GPU replicas out for bursts and back to zero when there is no work.

OPERATIONS

Focus on the app

Azure manages the host infrastructure; you manage model quality and behavior.

Serverless removes host management, not model operations or latency engineering.

Choose how much of the AI stack you want to own

These are operating models, not a universal ranking of price or performance.

HOSTED MODEL API

Consume a model

Fastest route to an API.
Less runtime control.
Model and service choices
set the constraints.

ACA SERVERLESS GPU

Own the container

Custom model and runtime.
Managed GPU infrastructure.
Replica-based scaling
with a zero-replica option.

AKS / GPU VMS

Own more compute

More scheduling and
infrastructure control.
More operational work.
Evaluate for advanced needs.

Choose ACA when container-level control matters more than cluster-level control.

Source: GPU overview | Workload profiles

You deliver the inference service. Azure runs the hosts.

Serverless GPU is a Consumption GPU workload profile inside a workload-profiles environment.

YOUR TEAM

- Container image, model weights and licenses
- Inference runtime, CUDA dependencies and API
- Scale limits, probes, identity and access
- Model quality, latency targets and telemetry

AZURE PLATFORM

- GPU-enabled infrastructure and host lifecycle
- Platform NVIDIA driver and runtime updates
- Replica orchestration and scaling execution
- Managed ingress and revision capabilities

Important: a legacy Consumption-only environment does NOT support serverless GPUs.

Source: GPU overview | Workload profiles

T4 or A100? Start with memory, then benchmark.

Model weights are only part of GPU memory use: leave room for runtime, activations and KV cache.

SELECTION FACTOR	NVIDIA T4	NVIDIA A100
GPU memory	**16 GB VRAM**	**80 GB HBM2/HBM2e**
Typical starting point	**Smaller inference models**	**Larger / demanding models**
Decision metric	**Cost per successful request**	**Latency and throughput**
Profile type	**Consumption-GPU-NC8as-T4**	**Consumption-GPU-NC24-A100**

Choose the smallest GPU that meets measured memory, throughput and latency requirements.

Source: GPU comparison | Workload profiles

Check region, environment and quota before building

GPU availability is not identical across regions or subscriptions.

1. **Select a supported region and GPU type**
   Documented examples: East US, Sweden Central and West US 3 offer both T4 and A100.

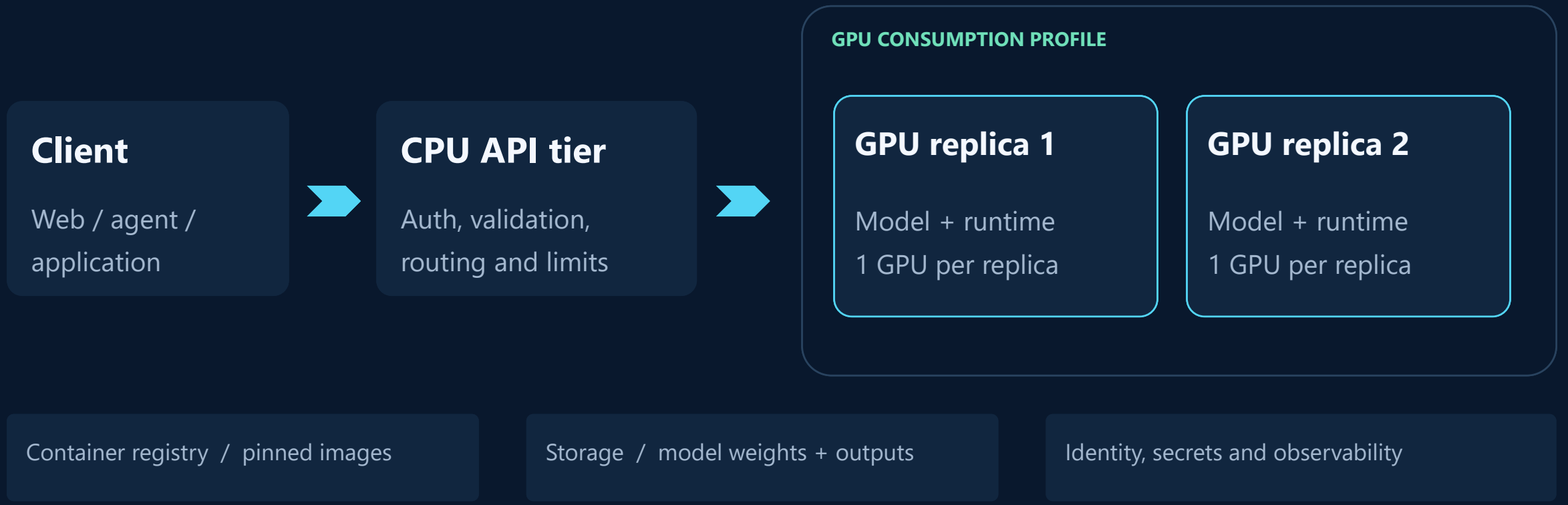
2. **Create a workload-profiles environment**
   Do this before requesting managed-environment GPU quota. Legacy Consumption-only is not supported.

3. **Inspect available GPU quota; request an increase if needed**
   Check the environment's Quota page. Quota is a limit, not a guarantee of immediate physical capacity.

West Europe T4 has an extra documented condition: create a new workload-profiles environment.

Separate the application tier from GPU inference

A conceptual online inference design; network and identity boundaries must be configured explicitly.



Source: GPU overview | Scaling

Scale replicas with demand, not one GPU per request

Illustrative behavior: actual response depends on rules, model startup, quota and regional capacity.



CONFIGURATION

Minimum replicas: 0

Maximum: set a safe cap

Trigger: HTTP or events

Tune concurrency by test

Scale-to-zero requires a wake-up signal. CPU/memory-only scaling is not a zero-to-one trigger.

Source: Scaling | GPU overview

Cold starts are a budget you must measure

Scale-to-zero trades idle compute cost for work on the next activation.

01
Provision
Obtain capacity

›

02
Image
Fetch / start image

›

03
Load
Read model weights

›

04
Warm up
Initialize the runtime

Reduce the work
Lean images, nearby registry, pre-staged weights; consider ACR Premium artifact streaming.

Protect the experience
Explicit startup/readiness probes; asynchronous UX or warm replicas when latency requires them.

No universal cold-start number: measure p50/p95 for the actual model, image and region.

Source: Cold starts | GPU overview | Health probes

Use asynchronous work for long-running generation

Keep interactive inference and durable background processing as distinct application contracts.

ONLINE PATH

Request -> inference -> response

Good for latency-bounded requests.
Use streaming only if the runtime supports it.
Handle activation delays and timeouts.
Bound concurrent work on each replica.

ASYNC PATH

Submit -> queue -> GPU worker

Return a task ID and status endpoint.
Persist results in object storage.
Use event-based activation and retries.
Make processing idempotent.

A queue absorbs bursts; it does not eliminate capacity limits, queue latency or failure handling.

From a container image to a GPU-backed endpoint

Walkthrough outline: deploy a small, licensed inference model that fits the selected GPU.

1. **Prepare the image**
   Pin runtime versions; test GPU access and model loading.
2. **Create the environment**
   Select a supported region and workload-profiles type.
3. **Check GPU quota**
   Inspect the environment quota; request more if needed.
4. **Choose GPU resources**
   In the Container tab, select GPU, then T4 or A100.
5. **Configure the app**
   Set identity, storage, ingress, probes and scale limits.
6. **Deploy and exercise**
   Test activation, inference, burst traffic and scale-in.

Use the same image digest, model revision and request set for repeatable performance comparisons.

Source: GPU overview | Health probes

Prove the behavior, not just a successful deployment

Record real measurements during the demo. No latency or throughput results are assumed here.

A / FUNCTIONAL

Model runs on GPU

Confirm GPU device access.
Check the model revision.
Validate response quality.

B / PERFORMANCE

Compare cold vs. warm

Record cold-start delay.
Measure warm p50 / p95.
Track tokens or results/sec.

C / ELASTICITY

Watch replicas scale

Observe zero-to-one startup.
Verify capped scale-out.
Confirm scale-in to zero.

Also test one failure: invalid input, model-load failure, saturation or an unavailable dependency.

Source: Scaling | Health probes | Billing

You pay for allocated GPU time, not GPU utilization

Serverless GPU replicas are always billed at active rates while running. There is no GPU idle discount.

ILLUSTRATIVE GPU-SECONDS ONLY

24 hours -> 2 hours allocated

86,400 -> 7,200 GPU-seconds

91.7% fewer allocated GPU-seconds

Estimate the whole system

GPU + CPU + memory meters

Billable external HTTP requests

Registry, storage and logs

Networking and feature charges

GPU cost = allocated GPU-seconds x applicable GPU rate; add all other billable meters.

This is not a bill estimate: startup, warm time, scale-in delay and rate differences change the outcome.

Source: Billing | Pricing

Managed infrastructure is not automatic data protection

Treat prompts, outputs, model files and telemetry as governed application data.

Identity first

Prefer managed identity for supported Azure access.

Scope permissions; keep secrets out of images.

Protect artifacts

Pin and scan images; verify model provenance.

Approve model licenses and storage locations.

Network deliberately

Choose internal or external ingress intentionally.

Authenticate callers and constrain data egress.

Govern data

Define retention and output access policies.

Redact sensitive prompts and results from logs.

Data can still leave via your code, outbound calls, mounts or logs. Configure and verify each path.

Source: GPU overview | Workload profiles

Measure the model, the service and the scaling loop

Collect application and runtime metrics explicitly; not every GPU metric is emitted by default.

USER EXPERIENCE

Did the request succeed?

End-to-end p50 / p95 latency
Time to first token or result
Errors, timeouts and retries
Queue wait and abandonment

GPU RUNTIME

Is the model healthy?

GPU memory and OOMs
Model-load / warm-up time
Throughput and batch size
Device and model revision

PLATFORM & COST

Is elasticity working?

Replica counts by revision
Activation and restart events
Quota / capacity failures
Allocated time and spend

Correlate requests, model version and replica/revision IDs so failures can be reproduced.

Source: Cold starts | Scaling | Health probes

Know the limits before committing the architecture

Scale-out means more single-GPU replicas, not a larger multi-GPU replica.

1. **One GPU per replica; no multi-GPU or fractional-GPU replicas**
   Plan memory fit on one GPU. More replicas do not automatically shard a large model.

2. **Only the first container in the app gets GPU access**
   Place the inference container first. Sharing a GPU profile does not mean sharing one replica.

3. **The platform GPU software stack evolves**
   Pin container dependencies and test driver/runtime compatibility before platform transitions.

VNet integration consumes one IP per Consumption GPU replica; include scale-out in subnet planning.

Source: GPU overview | Health probes

Use demand shape and runtime needs to make the call

Architectural guidance: validate with a workload-specific proof of concept.

WORKLOAD	INITIAL DIRECTION	WHAT TO VALIDATE
Bursty custom-model API	ACA serverless GPU	Memory fit + cold-start tolerance
Long image / media tasks	Queue + GPU worker	Durability + retry behavior
Steady high GPU utilization	Compare dedicated options	Measured total cost and utilization
Strict always-fast response	Warm capacity or another tier	Tail latency + availability needs
Distributed multi-GPU training	Evaluate AKS / Azure ML / VMs	Topology + scheduler requirements

Fit is a combination of control, GPU memory, demand variability, latency and operational effort.

Source: GPU overview | Workload profiles

Before launch: clear six acceptance gates

Define explicit pass/fail criteria before exposing a production endpoint.

CAPACITY
Region and environment verified;
quota covers planned peak replicas.

RESILIENCE
Timeouts, retries and saturation tested;
queue and output durability validated.

COMPATIBILITY
Pinned image and model tested;
GPU memory and CUDA fit confirmed.

SECURITY
Caller authorization verified;
data flows and license approved.

LATENCY
Cold and warm p95 measured;
startup / readiness probes verified.

COST
Replica and revision caps checked;
alerts and cost model ready.

A deployment that returns HTTP 200 is not yet a production-readiness test.

Source: GPU overview | Scaling | Billing | Health probes

Bring the model. Engineer the service.

Serverless GPU is most compelling when elasticity and container control both matter.

CONTROL

Own the model and runtime.

Azure manages GPU hosts.

ELASTICITY

Scale single-GPU replicas.

Measure startup and load.

ECONOMICS

Pay for allocated time.

Scale GPU replicas to zero.

Next: prove one model, one region and one representative traffic pattern.

Source: GPU overview | Billing

Official references

Microsoft documentation consulted 20 October 2026. Links below are clickable.

GPU overview
azure/container-apps/gpu-serverless-overview

GPU comparison
azure/container-apps/gpu-types

Workload profiles
azure/container-apps/workload-profiles-overview

Scaling
azure/container-apps/scale-app

Cold starts
azure/container-apps/cold-start

Billing
azure/container-apps/billing

Health probes
azure/container-apps/health-probes

Pricing
azure.microsoft.com/en-us/pricing/details/container-apps/

Recheck the live regional matrix, quota, software-stack guidance and prices before deployment.